

Thapar Connect -- Full CI/CD & Deployment Guide

A complete record of concepts, commands, and reasoning

Compiled for Daksh Kohli

July 2026

Introduction

Part 1: CI/CD Fundamentals

- What is CI/CD?

- The Mental Model

- How GitHub's Temporary VM Talks to a Real Server

- GitHub Secrets

Part 2: Understanding the Existing Setup

- Checking What Was Already on the Server

- What systemd Is, and Why It Matters Here

Part 3: Setting Up SSH Access for Automation

- Why a Separate SSH Key

- Installing the Public Key on the Server

- Verifying the New Key Works

- Setting Up Git Identity and Authentication (WSL side)

Part 4: Backend CI/CD Pipeline

- Creating the Workflow File

- The Final Backend Workflow

- Explaining Every Part

- Why Fixture Data Matters (and Why It's Safe)

- Types of Tests -- A Genuine Distinction

Part 5: Fixing systemd, Swap, and Server-Level Issues

- Adding Swap Space

- Fixing File Permissions for nginx

Part 6: Frontend Deployment

- Discovering the Real Tech Stack

- Building the Frontend

- Environment Variables in Vite -- A Critical Difference from Backend

- .env Files

Part 7: Setting Up nginx as a Reverse Proxy

- What a Reverse Proxy Actually Is

- Installing nginx

- The Configuration File

- Line-by-Line Explanation

- Enabling the Config

Part 8: The Long Debugging Journey -- Diagnosing a Mysterious

Redirect

- The Symptom

- Step 1 -- Ruling Out the Browser

- Step 2 -- Checking for Conflicting nginx Configs

- Step 3 -- The Redirect Persisted Anyway

- Step 4 -- Verbose curl to See Actual Response Headers

- Step 5 -- Searching AWS for the Load Balancer

- Step 6 -- The Actual Root Cause

- The Lesson

Part 9: Frontend CI/CD Pipeline

- The Workflow

- Why This Differs From the Backend Pipeline

- Debugging the First Real Run -- Lint Failures

- Part 10: Networking Deep Dive -- Ports, Firewalls, and Public Access
 - Security Groups vs. Network ACLs
 - Opening a Port in a Security Group
 - Verifying What's Actually Listening on a Port
 - The Full Request Journey (as finally configured)
- Part 11: HTTPS and Domains
 - Why HTTPS Requires a Domain
 - The Plan for When a Domain Is Assigned
- Part 12: Final Checklist
 - Infrastructure
 - Backend
 - Frontend
 - Reverse Proxy
 - Still Open (by design, not oversight)
- Closing Note

Introduction

This document is a complete record of setting up CI/CD and deploying the Thapar Connect application (frontend + backend) to a single AWS EC2 instance, with no domain, no Docker, using GitHub Actions. It covers every concept explained, every command run, why each step was necessary, and what problems came up along the way (including the debugging journey -- because that's often where the real learning happens).

Part 1: CI/CD Fundamentals

What is CI/CD?

CI (Continuous Integration) means that every time code is pushed, an automated system checks it -- running tests, linting, or builds -- so broken code is caught before it reaches production. This removes the need to manually test everything locally before every deployment.

CD (Continuous Deployment/Delivery) means that once those checks pass, the code is automatically shipped to the live server. Instead of manually SSHing in and running `git pull` and restarting a service, a machine does it for you, consistently, every time.

The Mental Model

```
Developer pushes code to GitHub
  |
  v
GitHub Actions triggers a "workflow"
  |
  v
Runs on a temporary GitHub-hosted virtual machine ("runner")
  |
  v
Step 1: Checkout code
Step 2: Install dependencies
Step 3: Run tests / build (this is the CI part)
  |
  v
```

If all steps pass -> Step 4: Deploy to the real server (this is the CD part)

How GitHub's Temporary VM Talks to a Real Server

This is the part that confuses most people starting out. GitHub Actions runs your workflow on a **throwaway virtual machine** that exists only for the duration of that one run, and is destroyed afterward. It has no knowledge of your real server unless you explicitly connect them.

Two common approaches:

1. **SSH-based (push model)** -- GitHub's runner SSHs into your real server using a private key you provide, then executes commands remotely (git pull, restart a service, etc.). This is simple and direct -- ideal for a single-server setup like this one.
2. **Pull-based (agent on server)** -- a tool running permanently on your server periodically checks GitHub for new deployments and pulls them itself. This is more relevant at a much larger scale (e.g. Kubernetes-based systems) and would be overkill here.

We used SSH-based deployment, since it matches a single EC2 instance perfectly.

GitHub Secrets

GitHub Actions has an encrypted secrets store (**Repo Settings -> Secrets and variables -> Actions**). This is where sensitive values live -- SSH private keys, API URLs, etc. -- so they never appear in the workflow file itself or in git history. Secrets are injected into the workflow's environment only at run-time, inside the temporary, isolated container, and are never permanently written anywhere.

Secrets added during this project: - EC2_HOST -- the server's Elastic IP - EC2_USER -- the SSH username (ubuntu) - EC2_SSH_KEY -- a dedicated private SSH key created solely for GitHub Actions to use - VITE_API_BASE_URL -- the backend's URL, injected into the frontend build

Part 2: Understanding the Existing Setup

Before writing any automation, the actual server and repo were inspected -- never guess at a config; always verify first.

Checking What Was Already on the Server

Connected via SSH and explored the directory structure:

```
ssh -i ~/.ssh/your-key.pem ubuntu@<ec2-ip>
cd ~/thapar-connect
ls
```

This revealed the repo already contained Frontend/, backend, and backend_new folders -- with backend_new being the actively used one.

Checked what was inside the backend folder:

```
ls -la
```

-la shows all files including hidden ones (dotfiles like .env), which is important because things like environment variable files, virtual environments, and git metadata don't show up with a plain ls.

Checked dependencies:

```
cat requirements.txt 2>/dev/null || echo "no requirements.txt"
```

This pattern (cmd || echo "fallback") means: run the first command, and only if it fails, run the fallback. Useful for "check if this exists" scripts.

Checked how the backend was currently being run:

```
ps aux | grep -i uvicorn
```

ps aux lists all running processes; piping through grep -i uvicorn filters to only lines containing "uvicorn" (case-insensitive, due to -i). This revealed the backend was running via uvicorn, and specifically confirmed it was bound to 127.0.0.1:8000 and 0.0.0.0:8000 on two separate processes (likely one leftover manual test process, and one real one).

Checked if a systemd service already existed:

```
sudo systemctl list-units --type=service | grep -i thapar
```

This confirmed a service called thapar-backend.service already existed and was active (running).

Viewed the actual service file:

```
cat /etc/systemd/system/thapar-backend.service
```

This showed the exact command systemd uses to start the backend, its working directory, and restart policy -- critical information needed before writing any CI/CD automation, since the automation needed to restart this exact service.

What systemd Is, and Why It Matters Here

systemd is Linux's service manager -- it's what starts, stops, restarts, and monitors long-running background processes (called "services" or "units"). Instead of manually running python api_server.py in a terminal (which dies the moment you close the terminal or the SSH session ends), systemd: - Starts the process automatically on server boot - Restarts it automatically if it crashes (Restart=always) - Lets you check its status, view its logs, and control it with simple commands like systemctl restart <name>

This is why the CI/CD pipeline's final deploy step is just sudo systemctl restart thapar-backend -- systemd handles all the actual process management; the pipeline just needs to tell it "reload with the new code."

Part 3: Setting Up SSH Access for Automation

Why a Separate SSH Key

The personal `.pem` key used to manually access the server should never be shared with GitHub Actions. Instead, a **dedicated key pair** was created solely for automated deployments -- this way, if it's ever compromised or needs to be revoked, personal access is unaffected.

```
ssh-keygen -t ed25519 -f ~/.ssh/github-deploy-key -C "github-actions-deploy" -N ""
```

Breaking this down: `-t ed25519` -- the key *type/algorithm*. Ed25519 is a modern, fast, secure elliptic-curve algorithm, preferred over older RSA keys for new setups. `-f ~/.ssh/github-deploy-key` -- the *filename* to save the key pair as. This creates two files: `github-deploy-key` (private -- never shared) and `github-deploy-key.pub` (public -- safe to share/install anywhere). `-C "github-actions-deploy"` -- a *comment*, just a human-readable label attached to the key, useful for identifying which key is which later. `-N ""` -- sets an *empty passphrase*. Normally SSH keys can be passphrase-protected for extra security, but GitHub Actions can't type a passphrase interactively during automated runs, so this must be blank for automation to work.

Installing the Public Key on the Server

Printed the public key to copy it:

```
cat ~/.ssh/github-deploy-key.pub
```

Then, while SSH'd into the server as the normal user, appended it to the list of authorized keys:

```
echo "ssh-ed25519 AAAA...(the actual key)..." >>
~/.ssh/authorized_keys
```

Concept: `authorized_keys` -- this file lists every public key allowed to log in as this particular user. It can hold multiple keys simultaneously - adding a new one doesn't remove or affect existing ones. This is why the original personal key kept working throughout.

Verifying the New Key Works

Tested from the local machine, before ever trusting GitHub with it:

```
ssh -i ~/.ssh/github-deploy-key ubuntu@<ec2-ip>
```

If this logs in without needing the old `.pem` file, the new key is correctly authorized.

Setting Up Git Identity and Authentication (WSL side)

Since git commits need an author identity attached, this had to be configured once:

```
git config --global user.name "Daksh Kohli"
git config --global user.email "kohlidaksh1707@gmail.com"
```

--global means this applies to every repo on this machine, not just one -- omit it to set a repo-specific identity instead.

Important lesson learned: GitHub removed password-based authentication for git operations in 2021. Typing your actual GitHub password when git prompts for one will *always* fail -- not because the password is wrong, but because that entire authentication method has been disabled. The replacement is a **Personal Access Token (PAT)**, used in place of a password.

Creating a PAT

1. GitHub -> profile picture -> Settings
2. Developer settings (bottom of left sidebar)
3. Personal access tokens -> Tokens (classic) -> Generate new token (classic)
4. Set an expiration (90 days was used here)
5. Check the repo scope (grants push/pull access)
6. **Also check workflow** -- this extra scope is specifically required if your push touches anything inside `.github/workflows/`. GitHub treats workflow files as more sensitive (since they can execute arbitrary code with access to secrets), so a plain repo-scoped token gets explicitly rejected with an error like: `> "refusing to allow a Personal Access Token to create or update workflow .github/workflows/deploy-backend.yml without workflow scope"`
7. Generate, and copy the token immediately -- GitHub shows it exactly once.

Using the PAT

When git push prompts for a password, paste the token instead. To avoid retyping it every time:

```
git config --global credential.helper store
```

This saves the token locally (in plaintext, at `~/.git-credentials`) after it's entered successfully once -- acceptable for a personal machine, not for a shared/public one.

A Note on Learning the Terminal

No developer memorizes every command from scratch. The real tools people rely on are: - **Up arrow** -- cycles through previous commands in your shell history - **Ctrl+R** -- fuzzy-searches your command history (type it, then start typing part of a remembered command) - **Aliases** -- shortcuts you define yourself for commands you use often, e.g. adding this to `~/.bashrc`: `bash alias ec2="ssh -i ~/.ssh/your-key.pem ubuntu@65.2.222.52"` then `source ~/.bashrc`, after which typing just `ec2` connects instantly. - A personal notes file/cheatsheet, built up over time, is completely normal practice -- not a sign of not "really" knowing something.

Part 4: Backend CI/CD Pipeline

Creating the Workflow File

GitHub Actions only recognizes workflow files placed in a specific path: `.github/workflows/`.

```
mkdir -p .github/workflows
nano .github/workflows/deploy-backend.yml
```

`mkdir -p` creates the folder (and any missing parent folders) without erroring if it already exists -- safe to run repeatedly.

The Final Backend Workflow

```
name: Deploy Backend

on:
  push:
    branches: [main]
    paths:
      - 'backend_new/**'

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend_new
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: python main.py
      - run: python test.py

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy over SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.EC2_HOST }
          username: ${ secrets.EC2_USER }
          key: ${ secrets.EC2_SSH_KEY }
          script: |
            cd /home/ubuntu/thapar-connect/backend_new
            git pull origin main
            source venv/bin/activate
            pip install -r requirements.txt
            sudo systemctl restart thapar-backend
```

Explaining Every Part

on: push: branches: [main] -- this workflow only triggers when commits are pushed directly to the main branch (not other branches, not on pull requests).

paths: - 'backend_new/'** -- a *path filter*. The workflow only fires if the push actually changed a file inside `backend_new/`. Since this repo has both frontend and backend code together, this prevents a frontend-only change from triggering an unnecessary (and pointless) backend redeploy, and vice versa. This is why pushing just the workflow file itself (which lives in `.github/workflows/`, not `backend_new/`) never triggers a run -- that's expected behavior, not a bug.

jobs: test: -- the first job. runs-on: ubuntu-latest means this job executes on a temporary, GitHub-provided Ubuntu virtual machine -- completely separate from the real EC2 server.

defaults: run: working-directory: backend_new -- every step in this job runs as if you'd first `cd backend_new`, so all subsequent paths are relative to that folder.

actions/checkout@v4 -- a pre-built "action" (reusable automation step) that clones your repository's code onto the temporary runner. Without this step, the runner has no files at all -- it starts completely empty.

actions/setup-python@v5 -- installs the specified Python version onto the runner.

pip install -r requirements.txt -- installs all Python dependencies listed in that file (fastapi, uvicorn, boto3, etc.), so the subsequent commands have everything they need to actually run.

python main.py -- this is the real, meaningful CI check for this project. It runs the actual scheduler logic against small "fixture" Excel files already committed to the repo (dummy student/course/room data used purely for testing, completely separate from real production data). This generates a fresh output/timetable.json -- entirely inside the temporary runner's filesystem, never touching the real server.

python test.py -- runs the project's real validation suite: 11 separate checks (no student clashes, no teacher clashes, no room clashes, no over-capacity rooms, etc.) against the freshly generated timetable. The script is written to `exit(0)` if everything passes and `exit(1)` if anything fails -- and GitHub Actions treats a non-zero exit code as a failed step, which automatically fails the entire job.

jobs: deploy: needs: test -- the second job. The `needs: test` line is the single most important concept in this whole pipeline: it makes the deploy job **wait for test to finish, and skip entirely if test fails**. This is what turns plain "push and auto-deploy" into a real CI/CD pipeline -- bad code physically cannot reach production, because the deploy step never even runs if the tests failed.

appleboy/ssh-action@v1.0.3 -- a community-built, reusable action that handles the entire SSH connection + remote command execution for you, so you don't have to hand-write raw SSH logic in bash yourself.

host, username, key -- pulled from GitHub Secrets (`${{ secrets.NAME }}` syntax), never hardcoded in the file, so nothing sensitive is visible in the repository.

The remote script: block -- this is the literal sequence of commands executed *on the real EC2 server* via SSH: `- cd /home/ubuntu/thapar-connect/backend_new` -- move into the project folder `- git pull origin main` -- fetch and apply the latest committed code `- source venv/bin/activate` -- activate the Python virtual environment (isolates this project's Python packages from the system's global Python) `- pip install -r requirements.txt` -- install any newly added dependencies `- sudo systemctl restart thapar-backend` -- restart the actual running service so it picks up the new code

Why Fixture Data Matters (and Why It's Safe)

The scheduler (`main.py`) requires 5 Excel files (students, curriculum, teacher names, teachers, rooms) to exist before it can run -- normally these get uploaded by an admin through the live application, which CI obviously can't do. The solution: **small, fake, dummy Excel files were already committed to git** under the `data/` folder, purely so automated tests have something realistic-shaped to run against.

Why this can never affect real production data: the CI test run happens entirely inside GitHub's temporary, isolated virtual machine. That machine: - Is spun up fresh for every single workflow run - Is completely destroyed the moment the workflow finishes - Has zero network connection to the real EC2 server

The only bridge between the two environments is the deploy job's SSH connection -- and that connection only ever runs `git pull` and a service restart, never touching output/ or any generated file. There is no code path by which a CI test run could overwrite the real, live output/timetable.json sitting on the actual server.

Types of Tests -- A Genuine Distinction

Unit tests check one small, isolated piece of logic -- a single function, with no dependencies on anything else. Example:

```
def test_slot_to_time_converts_correctly():
    assert slot_to_time(0) == "09:00"
```

Fast (milliseconds), and pinpoints exactly which function broke.

Integration tests (what `test.py` actually is) run the *entire* assembled system and check that the *combined final output* is correct as a whole. They catch bugs that only exist once separate pieces interact -- for example, two individually-correct functions producing a wrong result together. `test.py` specifically checks **emergent properties** of the whole generated timetable (no clashes, no capacity violations, etc.) -- properties that don't exist in any single function alone, only in the assembled result.

Why testing has to be deliberate, not automatic-by-default: the value of a test is proportional to the *cost of being wrong*. A scheduler silently double-booking a real professor is a genuinely damaging, embarrassing failure discovered by real people -- high cost, clearly worth testing. A tiny CSS misalignment on a page is low-cost, easily noticed and fixed -- not worth the same rigor. The actual judgment call is always: "what's the blast radius if this specific piece of logic is wrong, and does that justify the time spent testing it" -- not testing indiscriminately for its own sake.

Part 5: Fixing systemd, Swap, and Server-Level Issues

Adding Swap Space

The EC2 instance turned out to have only ~908MB of RAM (a `t3.micro`), and a Killed error appeared mid-build (the Linux **OOM killer** -- Out Of Memory killer -- forcibly terminates a process when

the system genuinely runs out of memory, to prevent a full system crash).

Fix -- add swap space, which lets the OS use disk space as overflow "virtual RAM" when physical RAM runs out (slower than real RAM, but prevents outright process termination):

```
sudo fallocate -l 1G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

- `fallocate -l 1G /swapfile` -- instantly creates a 1GB file (this "allocates" the space on disk without needing to actually write a gigabyte of zeros the slow way)
- `chmod 600 /swapfile` -- restricts the file's permissions so only the root user can read/write it (security best practice, since swap can technically contain sensitive data pulled from RAM)
- `mkswap /swapfile` -- formats that file specifically as swap space (similar to formatting a drive with a filesystem, but for swap's specific data structure)
- `swapon /swapfile` -- activates it immediately, for the current session
- `echo '...' | sudo tee -a /etc/fstab` -- appends a line to `/etc/fstab` (the file that lists what should be mounted automatically at every boot), so this swap file gets re-activated automatically after any future reboot, without needing to redo this manually

Verified with:

```
free -h
```

`-h` means "human-readable" (shows values in MB/GB instead of raw bytes). Before: Swap: 0B. After: Swap: 1.0Gi.

Cost note: this adds zero extra AWS charges -- it only uses space already inside the existing EBS disk volume, which is already being paid for.

Fixing File Permissions for nginx

When nginx tried to serve the frontend's built files, it returned a 500 Internal Server Error. Checking nginx's own error log revealed the real cause:

```
sudo tail -20 /var/log/nginx/error.log
```

This showed: `stat() "/home/ubuntu/.../dist/index.html" failed (13: Permission denied)`.

The concept: nginx's worker processes run as a dedicated, low-privilege system user (typically `www-data`), not as `ubuntu`. Home directories on Linux (`/home/ubuntu`) are usually locked down (permission mode `750`) so that *only* the owning user (and root) can even enter them -- meaning `www-data` was being blocked from even reaching the folder, let alone reading the files inside.

Fix:

```
sudo chmod o+x /home/ubuntu
sudo chmod o+x /home/ubuntu/thapar-connect
sudo chmod o+x /home/ubuntu/thapar-connect/Frontend
sudo chmod -R o+rX /home/ubuntu/thapar-connect/Frontend/dist
```

- `chmod o+x <folder>` -- grants "others" (any user not the owner and not in the owning group -- this includes `www-data`) **execute** permission specifically on a *directory*. For directories, "execute" permission doesn't mean "run it like a program" -- it means "you're allowed to enter/traverse through this folder." This was applied to each parent folder in the path, one level at a time, since traversal permission is required at *every* level of the path, not just the final destination.
- `chmod -R o+rX <folder>` -- applied recursively (-R, meaning "this folder and everything inside it, at every depth") to the actual `dist/` folder specifically. `r` grants read permission (so file *contents* can actually be served, not just "seen to exist"). Capital `X` (as opposed to lowercase `x`) is a special case: it grants execute/traversal permission *only* to subdirectories, not to regular files -- since a plain HTML/JS/CSS file never needs to be "executed," only read.

This is a minimal, safe fix -- it doesn't change file ownership, doesn't grant write access to anyone, and only opens the exact minimum path `nginx` genuinely needs.

Part 6: Frontend Deployment

Discovering the Real Tech Stack

Initial assumption was that the frontend was Next.js -- this turned out to be wrong, and was corrected by actually inspecting `package.json`:

```
cat package.json
```

This revealed scripts like `"dev": "vite", "build": "vite build"` -- confirming this is a **Vite + React + TypeScript** project (using `shadcn/ui` components), not Next.js. This is a good example of why inspecting the actual environment beats assuming based on general patterns.

Why this distinction matters enormously: Next.js (in its default mode) needs a persistent, running Node.js server process to handle server-side rendering. Vite, by contrast, builds down to **plain static files** -- HTML, CSS, and JS bundles that need no running process at all in production. This drastically simplified the deployment: no `systemd` service needed for the frontend, no Node process to keep alive -- just serve the built files directly from disk via `nginx`.

Building the Frontend

```
npm install
npm run build
```

`npm install` reads `package-lock.json` and downloads every dependency into a local `node_modules/` folder. `npm run build` executes the `build` script defined in `package.json` (which runs `vite build` under the hood) - - this bundles, minifies, and optimizes all the source code into a `dist/` folder containing the final, deployable static files.

Important distinction learned: `npm run dev` starts a live development server (with hot-reloading) that runs indefinitely and never finishes on its own -- this is *not* what you want for production,

and was mistakenly run once during this process, requiring Ctrl+C to stop it. `npm run build` is a one-shot command: it runs, prints progress, and exits cleanly once done.

Environment Variables in Vite -- A Critical Difference from Backend .env Files

Backend .env files (read by Python's `dotenv` library) are read live, every time the application starts -- you can change the file and just restart the service to pick up new values.

Vite environment variables work completely differently: they get baked directly into the compiled JavaScript bundle *at build time*. Once `npm run build` has run, the value is permanently embedded in the output files -- changing the .env file afterward does nothing until you rebuild.

This was discovered by inspecting the frontend's source code:

```
grep -rn "API_BASE_URL|localhost:8000|import.meta.env" src/lib/ | head -30
```

`grep -rn` searches recursively (`-r`) through a folder, showing line numbers (`-n`), for any of the given patterns (separated by `|`, meaning "or" in this `grep` syntax). This confirmed every single API file in the project consistently used the same pattern:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? "http://localhost:8000";
```

Meaning: if the environment variable isn't set at build time, it silently falls back to `localhost:8000` -- which would be completely broken in production, since "localhost" from a user's browser refers to *their own machine*, not the server.

Fix: created a dedicated production environment file:

```
echo 'VITE_API_BASE_URL=http://65.2.222.52:8080/api' > .env.production
```

Vite automatically uses `.env.production` values specifically when running a production build (`vite build`), and `.env.development` values when running the dev server (`vite dev`) -- using the `.production` suffix (rather than a generic `.env`) guarantees a dev/localhost URL can never accidentally end up in a real production build.

Then rebuilt:

```
rm -rf dist
npm run build
```

Removing `dist/` first ensured a completely fresh build, with no stale leftover files from a previous attempt.

Verified the correct URL actually got embedded into the compiled output:

```
grep -o "http://65.2.222.52:8080/api" dist/assets/*.js | head -1
```

`grep -o` prints only the matching text itself (not the whole line) -- a clean way to confirm a specific string exists somewhere inside the built JavaScript bundle.

Part 7: Setting Up nginx as a Reverse Proxy

What a Reverse Proxy Actually Is

Normally, a client (browser) connects directly to a server. A **reverse proxy** sits in between: the client only ever talks to the proxy, and the proxy decides, based on configured rules, which real backend service should actually handle each request -- then relays the response back. The client never needs to know which real service answered.

The word "reverse" specifically distinguishes this from a "forward proxy" (like a VPN), which acts on behalf of the *client* to hide the client's identity from servers -- a reverse proxy instead acts on behalf of the *server side*, managing and hiding the real backend infrastructure from the outside world.

Installing nginx

```
sudo apt install nginx -y
```

The Configuration File

```
sudo nano /etc/nginx/sites-available/thaparconnect
```

```
server {
    listen 8080;
    server_name _;
    root /home/ubuntu/thapar-connect/Frontend/dist;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        try_files $uri /index.html;
    }
}
```

Line-by-Line Explanation

listen 8080; -- this is the *only* port nginx accepts public connections on. This means nginx is the single point of entry for all traffic -- neither the frontend files nor the backend API are ever reachable directly from the internet.

server_name _; -- the underscore is a wildcard meaning "match any hostname" -- this works correctly even with just a bare IP address and no domain name, which is exactly the situation here.

root /home/.../dist; -- tells nginx where on disk to look for files when serving static content.

index index.html; -- when a request matches a directory rather than a specific file, serve index.html from that directory by default.

location /api/ { proxy_pass http://127.0.0.1:8000/; ... } -- this is a routing rule: any request whose path starts with /api/ is *not* served from disk. Instead, it gets forwarded to 127.0.0.1:8000 -- the backend's actual address. Since 127.0.0.1 (loopback) only accepts connections from the same physical machine, this means the backend is **never directly reachable from the public internet** -- only nginx, running on that same machine, can reach it. This is the "protected" route.

The four proxy_set_header lines forward important information about the *original* request through to the backend, since by default a proxied request would otherwise look to the backend like it came from nginx itself, not the real client: - Host \$host -- preserves the original hostname requested - X-Real-IP \$remote_addr -- passes along the real client's IP address - X-Forwarded-For \$proxy_add_x_forwarded_for -- a standard header listing the chain of proxies/IPs a request passed through - X-Forwarded-Proto \$scheme -- tells the backend whether the original request was HTTP or HTTPS

location / { try_files \$uri /index.html; } -- this is essential for any React Router (or similar client-side routing) application. Without it, refreshing the browser on a route like /admin/dashboard would return a 404, because no physical file named dashboard exists on disk -- the routing is handled entirely inside the JavaScript, client-side. This line tells nginx: "try to find a real file matching this exact request path first; if none exists, just serve index.html anyway" -- which lets the React app's own router take over and correctly render the right page.

Enabling the Config

```
sudo ln -sf /etc/nginx/sites-available/thaparconnect
/etc/nginx/sites-enabled/thaparconnect
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

- ln -sf creates a *symbolic link* (a shortcut/pointer) from sites-enabled/ to the actual config file in sites-available/. nginx only reads configs that are linked into sites-enabled/ -- keeping the real files in sites-available/ and only linking the ones currently active is the standard nginx convention, making it easy to enable/disable sites without deleting anything.
- -f forces the link creation even if one already exists at that path (overwrites it).
- rm -f /etc/nginx/sites-enabled/default removes the default placeholder site nginx ships with, which would otherwise potentially conflict.
- nginx -t -- **tests** the configuration syntax *without* actually applying it. This should always be run before reloading, since a typo here could otherwise take down anything currently running.
- systemctl reload nginx -- applies the new configuration without fully restarting the nginx process (a "reload" re-reads config files gracefully; a full "restart" would briefly drop all active connections).

Part 8: The Long Debugging Journey -- Diagnosing a Mysterious

Redirect

This section is included in detail because debugging methodology is itself a valuable skill, and this was a genuinely instructive (if frustrating) real-world example of chasing down a subtle issue.

The Symptom

Visiting the server's IP in a browser showed a completely unrelated login page ("Welcome to Fleet"), and curl showed a 301 Moved Permanently redirect to https://<ip>:443/.

Step 1 -- Ruling Out the Browser

```
curl http://<ip>/
```

Since curl (a command-line tool with no browser-side caching, HSTS, or auto-upgrade behavior) showed the exact same redirect, this immediately ruled out any browser-specific cause (like Chrome's "Always use secure connections" setting, or cached HSTS policy) -- proving the redirect was coming from somewhere in the actual network/server path, not the browser.

Step 2 -- Checking for Conflicting nginx Configs

```
sudo grep -rn "return 301|rewrite.*https" /etc/nginx/sites-enabled/  
/etc/nginx/conf.d/ 2>/dev/null  
ls -la /etc/nginx/sites-enabled/
```

This found a second, older, previously-forgotten config file (backend) still active in sites-enabled/, both using the same wildcard server_name _; as the newer config. When two nginx configs both claim the same wildcard hostname, nginx picks one somewhat arbitrarily (generally by alphabetical filename order) -- meaning the *old* config had been silently handling all traffic the entire time, proxying every request straight to the backend instead of serving the new frontend + reverse proxy setup.

Fix applied at the time:

```
sudo rm /etc/nginx/sites-enabled/backend  
sudo nginx -t  
sudo systemctl reload nginx
```

Step 3 -- The Redirect Persisted Anyway

Even after removing the conflicting config, the exact same 301 redirect still appeared. This meant the real cause was still undiscovered.

Step 4 -- Verbose curl to See Actual Response Headers

```
curl -v http://<ip>/
```

-v (verbose) shows the complete request and response, including every HTTP header -- not just the body. This revealed the critical clue:

Server: awselb/2.0

This header value is specific to **AWS Elastic Load Balancer** -- meaning something was intercepting the request *before it ever reached nginx at all*, since nginx would have identified itself as nginx/1.28.x, not awselb.

Step 5 -- Searching AWS for the Load Balancer

Checked EC2 -> **Load Balancing** -> **Load Balancers** in the AWS Console -- it showed "No load balancers" in that region. This was confusing at the time, since the evidence clearly pointed to a load balancer existing somewhere.

Step 6 -- The Actual Root Cause

After extensive checking of Security Groups, Network ACLs, and even creating a brand new port (8080) with fresh firewall rules to sidestep the issue -- the redirect *still* appeared, even on the new port, even when curling from the EC2 instance to its own public IP.

The real cause, finally identified: a simple digit transposition. The IP being tested (65.2.222.59) was **not actually this EC2 instance's IP at all** -- the real, correctly-assigned Elastic IP for this specific instance was 65.2.222.52. Every single test throughout this entire debugging session had been hitting a completely unrelated AWS resource (most likely another AWS customer's infrastructure entirely, since public IPv4 addresses get reused across AWS's whole customer base once released by a previous owner).

The moment testing switched to the correct IP:

```
curl -v http://65.2.222.52:8080/
```

The response immediately showed Server: nginx/1.28.3 (Ubuntu) -- confirming the actual, correct server, with no redirect at all.

The Lesson

This is a genuinely common category of real-world bug: **the code and configuration were correct the entire time; the input (which IP to test) was wrong**. No amount of reconfiguring nginx, Security Groups, or ports would ever have fixed a bug that didn't actually exist in that layer. The eventual fix was simply: verify the most basic assumption (which resource are we even talking to?) before continuing to add complexity on top of a broken foundation. This is why methodical, step-by-step verification (checking one layer at a time, starting from the most fundamental facts) -- rather than jumping straight to elaborate fixes -- is the right instinct when debugging anything unfamiliar.

Part 9: Frontend CI/CD Pipeline

The Workflow

```
name: Deploy Frontend

on:
  push:
    branches: [main]
    paths:
      - 'Frontend/**'

jobs:
  test:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: Frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
      - name: Create env file
        run: echo "VITE_API_BASE_URL=${{ secrets.VITE_API_BASE_URL
    }}" > .env.production
      - run: npm ci
      - run: npm run lint || true
      - run: npm run test
      - run: npm run build

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
      - name: Create env file
        working-directory: Frontend
        run: echo "VITE_API_BASE_URL=${{ secrets.VITE_API_BASE_URL
    }}" > .env.production
      - name: Install and build
        working-directory: Frontend
        run: |
          npm ci
          npm run build
      - name: Deploy dist/ to EC2
        uses: appleboy/scp-action@v0.1.7
        with:
          host: ${ secrets.EC2_HOST }
          username: ${ secrets.EC2_USER }
          key: ${ secrets.EC2_SSH_KEY }
          source: "Frontend/dist/*"
          target: "/home/ubuntu/thapar-connect/Frontend/dist_new"
          strip_components: 2
      - name: Swap in new build atomically
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.EC2_HOST }
          username: ${ secrets.EC2_USER }
          key: ${ secrets.EC2_SSH_KEY }
          script: |
            cd /home/ubuntu/thapar-connect/Frontend
```

```
rm -rf dist_old
mv dist dist_old
mv dist_new dist
sudo chmod o+rX -R dist
```

Why This Differs From the Backend Pipeline

The backend pipeline uses `git pull` directly on the server, because the backend is Python source code that needs to exist on the server and be executed there by `uvicorn`.

The frontend is different: the server never needs the *source code* at all -- it only ever needs the final, already-built static files. So instead of pulling source and rebuilding *on* the resource-constrained EC2 instance (risking the same out-of-memory crash seen earlier), the build happens on GitHub's more powerful temporary runner, and only the finished `dist/` folder is transferred over.

`npm ci` (used instead of `npm install`) -- installs dependencies strictly according to `package-lock.json`, without ever modifying that lock file. This is the standard choice for CI environments specifically, since it guarantees perfectly reproducible installs and is generally faster than `npm install`.

Why the build happens twice (once in test, once in deploy) -- each GitHub Actions job runs on a completely separate, isolated temporary machine; files don't automatically carry over between jobs unless explicitly passed along via "artifacts." The test job's build exists purely to verify the code compiles cleanly (a quality gate) and its output is discarded; the deploy job performs its own fresh build specifically to produce the files that actually get shipped.

appleboy/scp-action -- a reusable action that copies files from the GitHub runner to a remote server over SCP (Secure Copy, an SSH-based file transfer protocol), analogous to how `appleboy/ssh-action` executes remote commands.

strip_components: 2 -- since the source path is `Frontend/dist/*`, this strips the leading `Frontend/dist/` portion from each file's path during transfer, so files land directly inside the target folder rather than being nested inside extra subfolders matching the source structure.

The "atomic swap" pattern (`dist -> dist_old`, `dist_new -> dist`) -- this exists specifically to avoid users ever hitting a half-updated site mid-deploy. If files were copied directly into the live `dist/` folder while `nginx` was actively serving requests, someone could theoretically load a page that mixes old and new file versions (e.g. an old `index.html` referencing a JS bundle filename that was already replaced). By building the complete new version in a separate folder first, and only "flipping" folder names at the very end (an operation that's nearly instantaneous), `nginx` always sees either the complete old version or the complete new version -- never a partially-updated mix.

Debugging the First Real Run -- Lint Failures

The very first real test run failed at the `npm run lint` step, with output showing dozens of warnings and two hard errors. Investigation revealed these were **pre-existing issues inside shadcn/ui's own**

auto-generated component files (`command.tsx`, `textarea.tsx`, etc.) -- code that was never written or reviewed by the project's own developers, and had simply never been checked by a linter before this pipeline existed.

The specific issues found: - Most were `react-refresh/only-export-components` warnings -- these only affect the developer experience during local development (specifically, Vite's Hot Module Replacement behavior when editing a file that exports both a component and a non-component value). They have **zero effect on production behavior**. - Two were `@typescript-eslint/no-empty-object-type` errors -- flagging an interface like `interface X extends Y {}` that adds nothing over just using `Y` directly. This is a valid but purely cosmetic/maintainability nitpick with **no possible runtime impact**.

Decision made: rather than spend time modifying auto-generated library boilerplate, the lint step was made non-blocking:

```
- run: npm run lint || true
```

The `|| true` pattern means: run the command, and regardless of whether it succeeds or fails, report success to the shell/pipeline. The lint output is still printed in full in the logs (so issues remain visible for anyone who wants to review them), but a failing lint check no longer prevents the deploy job from running.

This was the correct tradeoff here specifically because none of the flagged issues could cause an actual functional bug -- if any of them had represented a genuine correctness risk, blocking the pipeline until fixed would have been the right call instead.

Part 10: Networking Deep Dive -- Ports, Firewalls, and Public Access

Security Groups vs. Network ACLs

AWS has two separate, independent layers of network filtering, both of which must allow traffic for it to actually get through:

- **Security Groups** -- attached directly to an EC2 instance (or a specific network interface). Generally operate as "allow lists" -- by default, everything is denied, and you explicitly add rules to permit specific ports/sources.
- **Network ACLs (NACLs)** -- operate at the *subnet* level (a broader network boundary that can contain multiple instances), and can have both explicit ALLOW and explicit DENY rules, evaluated in a specific numbered order.

During the port 8080 troubleshooting, the correct debugging order was: first confirm the Security Group allows the port, then (if still blocked) check the Network ACL -- since either layer independently blocking traffic produces the same symptom (a connection timeout) from the outside.

Opening a Port in a Security Group

EC2 -> Instances -> select instance -> Security tab -> click the security group -> Edit inbound rules -> Add rule: - Type: Custom TCP - Port range: (the specific port, e.g. 8080) - Source: 0.0.0.0/0 (meaning "any IPv4 address on the internet" -- appropriate for public-facing testing, though a real production setup might restrict this further)

Verifying What's Actually Listening on a Port

```
sudo ss -tlnp | grep ':8080'
```

- `ss` -- a modern Linux utility for inspecting network sockets and connections (a more modern replacement for the older `netstat` command)
- `-t` -- show only TCP connections
- `-l` -- show only *listening* sockets (i.e., things actively waiting for incoming connections, not established/active connections)
- `-n` -- show numeric addresses/ports rather than trying to resolve them to hostnames (faster, and avoids DNS lookup delays)
- `-p` -- show the *process* that owns each socket (requires `sudo` to see process details for services not owned by your own user)

This command was used repeatedly throughout the session to definitively confirm exactly what was bound to which port -- an essential, authoritative source of truth that cuts through any confusion caused by browser caching, wrong IPs, or misremembered configuration.

The Full Request Journey (as finally configured)

```
Browser/curl request to http://65.2.222.52:8080/some-path
|
v
AWS Security Group (checks: is port 8080 allowed from this source? -
> yes)
|
v
nginx (the only process actually listening on port 8080)
|
+--- if path starts with /api/ ---> proxied to backend at
127.0.0.1:8000
|
|                                     (FastAPI/uvicorn,
reachable ONLY                       from the same machine
|                                     -- never
|                                     directly from the
internet)
|
+--- otherwise ---> served directly from
/home/ubuntu/.../Frontend/dist
                        (the built React app's static files)
```

Key security property worth understanding: even though the Security Group happens to also have port 8000 open (a leftover from earlier manual testing), the backend process itself is bound specifically to 127.0.0.1:8000 (loopback-only) rather than 0.0.0.0:8000 (all interfaces). This means an open firewall port and an actually-reachable service are two independent things -- the firewall being

open doesn't guarantee external traffic can complete a connection, since the operating system itself won't accept an external connection destined for a loopback-only-bound service. Both layers matter independently.

Part 11: HTTPS and Domains

Why HTTPS Requires a Domain

Certificate Authorities (like Let's Encrypt, the standard free option) issue TLS/SSL certificates specifically to *verified domain names* -- the entire security model of a certificate rests on proving "whoever requested this certificate genuinely controls this domain." Raw IP addresses don't have an equivalent, standard ownership-verification mechanism, and get reassigned between different customers constantly -- so mainstream Certificate Authorities generally will not issue certificates for bare IP addresses at all.

This is why, for the current stage of this project (no domain assigned yet), plain `http://` access is the correct and expected state -- not a mistake or something to be worked around. For a genuine production system (as this one is intended to become, serving 1000+ real users), waiting for proper domain-based HTTPS is the right call, rather than reaching for a workaround (like a third-party tunneling service) that wouldn't be appropriate at this scale or seriousness of deployment.

The Plan for When a Domain Is Assigned

1. Whoever manages DNS creates an **A record** -- a DNS record type that maps a domain name directly to an IPv4 address -- pointing the domain (or a subdomain) at `65.2.222.52`.
2. Install Certbot, the standard free tool for obtaining and auto-renewing Let's Encrypt certificates, along with its nginx-specific plugin:

```
sudo apt install certbot python3-certbot-nginx -y
```

3. Run:

```
sudo certbot --nginx -d yourdomain.com
```

Certbot automatically detects the existing nginx configuration, obtains a valid certificate, edits the nginx config to add the correct SSL/HTTPS block, and -- critically -- sets up automatic renewal (Let's Encrypt certificates expire every 90 days; Certbot installs a scheduled task that renews them automatically well before expiry, so this isn't an ongoing manual chore).

4. At this point, it would also make sense to move nginx from port 8080 back to the standard port 80 (and add port 443 for HTTPS), so the final URL is clean (`https://yourdomain.com`) with no port number required.

Part 12: Final Checklist

Infrastructure

- EC2 instance running, with a permanent Elastic IP (65.2.222.52) that survives reboots
- 1GB swap space added, preventing out-of-memory build failures
- Security Group configured with the necessary open ports

Backend

- Running via a systemd service (`thapar-backend.service`) on port 8000, bound to `127.0.0.1` only
- Full CI/CD: every push to `backend_new/` triggers the real scheduler plus all 11 domain-correctness tests, and only deploys via SSH if every test passes

Frontend

- Built with Vite into static files, correctly configured with the production backend URL baked in at build time
- Served by nginx on port 8080, with correct file permissions for the nginx worker user to actually read the files
- Full CI/CD: every push to `Frontend/` lints (non-blocking), runs any existing tests, builds, and deploys the built files via SCP with an atomic folder-swap to avoid serving a half-updated site

Reverse Proxy

- nginx is the sole public-facing entry point; the backend API is proxied through `/api/`, remaining otherwise unreachable directly from the internet
- Client-side routing (React Router) correctly handled via `try_files` fallback to `index.html`

Still Open (by design, not oversight)

- HTTPS -- correctly deferred until a real domain is provided; the plan for that moment is fully documented above
 - An application-level admin login bug -- identified as a code issue, unrelated to any infrastructure, to be debugged separately
 - Minor cosmetic lint issues in third-party generated component files -- safe to leave as-is or fix at leisure
-

Closing Note

Everything in this document reflects an approach worth internalizing beyond just this one project: inspect before configuring, verify one layer at a time when debugging, understand *why* a fix works rather than just applying it, and treat "the code is fine but I'm testing the wrong thing" as a genuinely common and important failure mode to check for early, before assuming something is broken at a deeper level than it actually is.